

## 程序设计 II 第 16 周 题目汇总（完整版 · STL）

来源：Matrix · 课程「程序设计 II（25 级 曾坤 计算机学院）」含：完整题面、提交文件、平台提供的初始文件（.hpp/main.cpp 等）。选择题含标准答案。

共 7 项：课后 1<sub>4</sub>（选做）、课堂 1<sub>2</sub>、理论题（20 题，选做）

---

### 25 程设II-周 16-课后 1（选做）

类型：编程题

题面

#### 查找和替换模式

给定单词列表 words 和模式 pattern。若存在字母排列 p（字母到字母的双射），将模式中每个字母 x 替换为 p(x) 后得到该单词，则单词与模式匹配。返回 words 中与给定模式匹配的单词（按输入顺序），无匹配输出 empty。

示例：words = ["abc","deq","mee","aqq","dkd","ccc"], pattern = "abb" → ["mee","aqq"]。"ccc" 不匹配，因为 a、b 映射到同一字母不是排列。注意：pattern.length = words[i].length。

**输入：**第一行单词列表（空格分隔）；第二行模式。**输出：**匹配单词（空格分隔），无则 empty。

输入: abc deq mee aqq dkd ccc / abb 输出: mee aqq

输入: a b c / a 输出: a b c

**提交文件：**Match.cpp

**平台提供的初始文件**

文件 Match.h：

```
#ifndef MATCH_H
#define MATCH_H
```

```
#include <iostream>
#include <vector>
#include <string>
using namespace std;
```

```
class Solution {
public:
    vector<string> findAndReplacePattern(vector<string>& words, string pattern);
};
```

```
#endif
```

文件 main.cpp :

```
#include "Match.h"
#include <vector>
#include <string>
#include <sstream>
using namespace std;

int main() {
    Solution solution;
    vector<string> words;
    string pattern;
    string input;

    getline(cin, input);
    istringstream iss(input);
    string word;
    while (iss >> word) {
        words.push_back(word);
    }

    cin >> pattern;

    vector<string> result = solution.findAndReplacePattern(words, pattern);

    if (result.empty()) {
        cout << "empty" << endl;
    } else {
        for (const auto& w : result) {
            cout << w << " ";
        }
        cout << endl;
    }

    return 0;
}
```

---

## 25 程设II-周 16-课后 2 (选做)

类型：编程题

## 题面

### 最大平均通过率

$n$  个班级， $\text{classes}[i] = [\text{pass}_i, \text{total}_i]$ 。额外  $\text{extraStudents}$  个必过学生分配到各班，使平均通过率最大（平均通过率 = 各班通过率之和  $\div$  班级数）。输出最大平均通过率。

输入: 2 / 3 / 1 2 / 3 5 / 2 2 输出: 0.783

输入: 4 / 4 / 2 4 / 3 9 / 4 5 / 2 10 输出: 0.535

数据范围： $1 \leq n \leq 1e5$ ， $1 \leq \text{pass}_i \leq \text{total}_i \leq 1e5$ ， $1 \leq \text{extraStudents} \leq 1e5$ 。

提交文件：MaxPassRatio.cpp

### 平台提供的初始文件

文件 main.cpp：

```
#include <iostream>
#include <vector>
#include "MaxPassRatio.hpp"
using namespace std;

int main() {
    int extraStudents, n;
    cin >> extraStudents >> n;

    vector<vector<int>>> classes(n, vector<int>(2));
    for (int i = 0; i < n; i++) {
        cin >> classes[i][0] >> classes[i][1];
    }

    double res = maxPassRatio(classes, extraStudents);
    cout << fixed << setprecision(3) << res;

    return 0;
}
```

文件 MaxPassRatio.hpp：

```
#ifndef MAXPASSRATIO_H
#define MAXPASSRATIO_H

#include <iostream>
#include <vector>
#include <queue>
#include <tuple>
#include <iomanip>

using namespace std;
```

```
double maxPassRatio(vector<vector<int>>& classes, int extraStudents);
```

```
#endif
```

---

## 25 程设II-周 16-课后 3（选做）

类型：编程题

题面

### 文本编辑器

实现 TextEditor：append(const string&)（末尾加行，记录以便撤销，新 append 清空 redo 历史）、undo()、redo()、getDocument() const（返回 vector<string> 或 list<string>）。提示：用 list<string> 存文档，两个 list<string> 作 undoStack/redoStack。

提交文件：TextEditor.cpp

平台提供的初始文件

文件 TextEditor.h：

```
#ifndef TEXTEDITOR_H
#define TEXTEDITOR_H

#include <string>
#include <list>
#include <vector>

class TextEditor {
private:
    std::list<std::string> document_lines_;
    std::list<std::string> undo_stack_;
    std::list<std::string> redo_stack_;

public:
    TextEditor() = default;

    void append(const std::string& line);
    void undo();
    void redo();
    std::vector<std::string> getDocument() const;
    void printDocument() const;
};

#endif // TEXTEDITOR_H
```

文件 main.cpp：

```
#include "TextEditor.h"
#include <iostream>
int main() {
    TextEditor editor;
    std::string a,b,c;
    std::getline(std::cin, a);
    std::getline(std::cin, b);
    std::getline(std::cin, c);
    editor.append(a);
    editor.append(b);
    editor.printDocument();

    editor.undo();
    editor.printDocument();

    editor.undo();
    editor.printDocument();

    editor.undo();
    editor.printDocument();

    editor.redo();
    editor.printDocument();

    editor.redo();
    editor.printDocument();

    editor.redo();
    editor.printDocument();

    editor.append(c);
    editor.printDocument();

    editor.redo();
    editor.printDocument();

    editor.undo();
    editor.printDocument();
    return 0;
}
```

---

## 25 程设II-周 16-课后 4（选做）

类型：编程题

## 题面

### 字谜猎手 (Anagram Hunter)

实现 `findAnagrams`，将字典中的字谜（字母相同顺序不同）分组。键 = 单词排序后的字符串（规范化形式），值 = `vector<string>`。组间按规范化串字典序升序；组内按首次出现顺序；单个词不输出；无字谜组输出 没有找到字谜组。

输入: listen silent enlist rat tar art banana

输出: rat, tar, art / listen, silent, enlist

提交文件：anagram\_utils.cpp

### 平台提供的初始文件

文件 anagram\_utils.h：

```
#ifndef ANAGRAM_UTILS_H
#define ANAGRAM_UTILS_H
```

```
#include <string>
#include <vector>
#include <map>
```

```
std::string getCanonicalForm(const std::string& word);
```

```
std::map<std::string, std::vector<std::string>> findAnagrams(const
std::vector<std::string>& dictionary);
```

```
void printAnagramGroups(const std::map<std::string, std::vector<std::string>>&
anagram_groups);
```

```
#endif
```

文件 main.cpp：

```
#include "anagram_utils.h"
#include <iostream>
#include <vector>
#include <string>
```

```
int main() {
    std::vector<std::string> dictionary;
    std::string word;

    while (std::cin >> word) {
        if (word.empty()) {
            continue;
        }
        dictionary.push_back(word);
    }
}
```

```
    auto anagram_groups = findAnagrams(dictionary);
    printAnagramGroups(anagram_groups);

    return 0;
}
```

---

## 25 程设II-周 16-课堂 1

类型：编程题

题面

### 只出现一次的数字

非空整数数组 `nums`，除一个元素只出现一次外其余均出现两次，找出那个元素。（经典异或解法）

输入: 3 / 2 2 1 输出: 1  
输入: 5 / 4 1 2 1 2 输出: 4  
输入: 1 / 1 输出: 1

提交文件：Num.cpp

平台提供的初始文件

文件 main.cpp：

```
#include "Num.h"
#include <iostream>
#include <vector>
using namespace std;

int main() {
    int n;
    cin >> n;

    vector<int> nums(n);

    for (int i = 0; i < n; ++i) {
        cin >> nums[i];
    }

    Solution sol;
    int result = sol.singleNumber(nums);
    cout << result << endl;

    return 0;
}
```

文件 Num.h :

```
#ifndef NUM_H
#define NUM_H

#include <iostream>
#include <vector>
using namespace std;

class Solution {
public:
    int singleNumber(vector<int>& nums);
};

#endif
```

---

## 25 程设II-周 16-课堂 2

类型：编程题

题面

### 智能书架接口设计

设计“智能书架” SmartBookshelf 管理 Book (title/author/isbn/publicationYear , isbn 唯一标识) 。实现：

1. addBook(Book)：ISBN 重复失败输出 ISBN 'book.isbn' exists 返回 false；空 ISBN 输出 empty ISBN！；成功输出 addBook success 返回 true。
2. removeBook(const string& isbn)：空 ISBN 输出 empty ISBN！ 返回 false；未找到输出 ISBN '对应 ISBN' not found 返回 false；成功输出 removeBook success 返回 true。
3. findBookByISBN(const string& isbn)：空 ISBN 输出 empty ISBN！ 返回 nullptr；找到输出 ISBN '对应 ISBN' title:对应书名 返回 const Book\*；未找到输出 ISBN '对应 isbn' not found 返回 nullptr。

提交文件：SmartBookshelf.cpp

### 平台提供的初始文件

文件 main.cpp：

```
#include <iostream>
#include <vector>
#include <string>
#include <algorithm>

#include "SmartBookshelf.h"
```



```

void printBook(const Book* book, const std::string& prefix = "") {
    if (book) {
        std::cout << prefix << "Title: " << book->title
            << ", Author: " << book->author
            << ", ISBN: " << book->isbn
            << ", Year: " << book->publicationYear << std::endl;
    } else {
        std::cout << prefix << "Book not found." << std::endl;
    }
}

```

```

void printBooksVector(const std::vector<Book>& books) {
    if (books.empty()) {
        std::cout << "No books to display." << std::endl;
    } else {
        for (const auto& book : books) {
            printBook(&book);
        }
    }
}

```

```

int main() {
    SmartBookshelf shelf;

    std::string name1,name2,name3;
    std::string writer1,writer2,writer3;
    std::string isbn1,isbn2,isbn3;
    int year1,year2,year3;
    std::cin>>name1>>writer1>>isbn1>>year1;
    std::cin>>name2>>writer2>>isbn2>>year2;
    std::cin>>name3>>writer3>>isbn3>>year3;
    Book b1 = {name1, writer1, isbn1, year1};
    Book b2 = {name2, writer2, isbn2, year2};
    Book b3 = {name3, writer3, isbn3, year3};

    shelf.addBook(b1);
    shelf.addBook(b2);
    shelf.addBook(b3);
    std::cout << shelf.getBookCount() << std::endl;

    const Book* found_b1 = shelf.findBookByISBN(isbn1);
    const Book* not_found_book = shelf.findBookByISBN("000-0000000000");

    shelf.removeBook(isbn1);
    std::cout << shelf.getBookCount() << std::endl;
    shelf.removeBook("111-1111111111");
    std::cout << shelf.getBookCount() << std::endl;
}

```

```

    const Book* find_b2_after_removal = shelf.findBookByISBN(isbn1);
    printBook(find_b2_after_removal);

    Book b_empty_isbn = {"Empty Test", "Test Author", "", 2024};
    shelf.addBook(b_empty_isbn);
    shelf.findBookByISBN("");
    shelf.removeBook("");

    return 0;
}

```

文件 SmartBookshelf.h :

```

#ifndef SMART_BOOKSHELF_HPP
#define SMART_BOOKSHELF_HPP

#include <string>
#include <vector>
#include <map>

struct Book {
    std::string title;
    std::string author;
    std::string isbn;
    int publicationYear;

    bool operator==(const Book& other) const {
        return isbn == other.isbn;
    }
};

class SmartBookshelf {
private:
    std::map<std::string, Book> booksByIsbn_;
public:
    SmartBookshelf();
    ~SmartBookshelf();

    bool addBook(const Book& book);
    bool removeBook(const std::string& isbn);
    const Book* findBookByISBN(const std::string& isbn) const;
    size_t getBookCount() const;
};

#endif

```

---

## 25 程设II-周 16-理论题（选做，选择题，20 题）

**第 1 题** 哪个容器存储唯一元素且可快速查找？ A. vector B. deque C. map D. set **答案：D**（基于红黑树， $O(\log n)$ ）

**第 2 题** list 容器的主要优势？ A. 随机访问 B. 高效的插入和删除 C. 可排序性 D. 固定大小 **答案：B**

**第 3 题** stack 的主要特征？ A. 随机访问 B. 先进后出（FILO） C. 双端队列 D. 无序集合 **答案：B**

**第 4 题** unordered\_set 的查找复杂度？ A.  $O(n)$  B.  $O(\log n)$  C.  $O(1)$  D.  $O(n^2)$  **答案：C**（哈希表）

**第 5 题** 如何访问 queue 的最后一个元素？ A. front B. back C. 不允许 D. pop **答案：B**

**第 6 题** priority\_queue 默认实现？ A. 最小堆 B. 最大堆 C. FIFO D. 无序集合 **答案：B**

**第 7 题** 哪个容器以动态数组形式存储且两端高效？ A. list B. deque C. stack D. queue **答案：B**

**第 8 题** std::map 中 m.find(k) 查找不存在的键返回？ A. nullptr B. m.begin() C. m.end() D. 报错 **答案：C**

**第 9 题** 下列哪个不是 STL 算法的特性？ A. 通用性 B. 可移植性 C. 高效性 D. 非重载性 **答案：D**

**第 10 题** 哪个容器适合频繁在两端插入删除？ A. vector B. list C. deque D. set **答案：C**

**第 11 题** 哪个容器不使用动态内存分配？ A. vector B. array C. list D. deque **答案：B**（大小编译期确定）

**第 12 题** 哪个容器在内存中不连续存储？ A. vector B. array C. list D. deque **答案：C**

**第 13 题** copy 算法主要用于？ A. 复制容器 B. 将一个范围的元素复制到另一个范围 C. 创建深拷贝 D. 复制单个元素 **答案：B**

**第 14 题** 哪个容器适合频繁插入删除但不需随机访问？ A. vector B. list C. array D. deque **答案：B**

**第 15 题** 哪个容器插入元素时可能导致迭代器失效？ A. list B. set C. vector D. map **答案：C**（重新分配内存时全部失效）

**第 16 题** remove 算法实际上做了什么？ A. 从容器删除元素 B. 将要删除的元素移到末尾 C. 标记要删除 D. 替换 **答案：B**（remove-erase 惯用法，需配合 erase）

**第 17 题** 哪个容器提供常数时间随机访问？ A. list B. vector C. map D. set **答案：B**

**第 18 题** generate 算法主要用于？ A. 生成随机数 B. 使用函数生成容器元素 C. 创建新容器 D. 复制元素 **答案：B**

**第 19 题** 哪个容器内存使用最高效？ A. vector B. list C. deque D. array **答案：D**

**第 20 题** reverse 算法主要用于？ A. 反转容器 B. 反转指定范围内元素顺序 C. 反转单个元素 D. 反转迭代器方向 **答案：B**

---